PARYAYA

RunPod Whisper Fine-Tuning Guide

Complete step-by-step: accounts → GPU pod → train → download → deploy

~$10–20 total cost	~3–6 hrs training (unattended)	A100 PCIe 40 GB

What This Guide Covers

- Fine-tune OpenAI Whisper medium (244M params) on Google FLEURS Nepali (ne_np)
- FLEURS is free and public — no account, no token, no terms acceptance needed
- No custom model architecture — uses HuggingFace Transformers Seq2SeqTrainer
- After training: point Paryaya API at checkpoint with ASR_BACKEND=whisper

Before: ASR_BACKEND=paryaya → custom conformer (untrained) | After: ASR_BACKEND=whisper → Whisper medium fine-tuned on real Nepali

TABLE OF CONTENTS

3

5

7

9

10

13

15

16

19

20

PART 0 — BEFORE YOU START

Complete these two steps before launching a RunPod pod.

■ Good news: we use Google FLEURS (ne_np) as the training dataset. FLEURS is completely free and public — no HuggingFace account, no token, and no terms acceptance required. Just run the training script and it downloads automatically.

0.1 Create a Weights & Biases Account

~5 minutes (optional)

~5 minutes (optional)

Why FLEURS instead of Common Voice?

- Mozilla moved all Common Voice datasets off HuggingFace in October 2025
- Google FLEURS (ne_np) is still on HuggingFace, free, and requires no login
- FLEURS has ~3,000 high-quality Nepali training clips — sufficient for fine-tuning
- Training takes 2-3 hours on A100 vs 6-8 hours for Common Voice (smaller dataset)

0.1 Create a Weights & Biases Account

~5 minutes (optional)

~5 minutes (optional)

■ W&B; gives you real-time loss and WER charts you can watch from your phone without keeping SSH open. Free tier is enough. Skip only if you want no monitoring.

- Go to **wandb.ai** and click **Sign up**
- Once logged in, go to **wandb.ai/settings**
- Scroll to **API keys** → click **New key** → copy it

Your W&B; key looks like:

```
abcdef1234567890abcdef1234567890abcdef12
```

0.2 Verify Your SSH Key Exists

~5 minutes

~5 minutes

You need an SSH key to connect to RunPod. Check if you have one:

```
# Run on your Mac terminal:
ls ~/.ssh/id_ed25519.pub
```

If the file exists — skip to "Copy your public key" below.

If you get "No such file or directory" — create one:

```
ssh-keygen -t ed25519 -C "paryaya-runpod"
# Press Enter three times (accept defaults, no passphrase)
```

Copy Your Public Key

You will paste this into RunPod in Part 1:

```
cat ~/.ssh/id_ed25519.pub
# Select all output and copy it
```

The output looks like:

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI... paryaya-runpod
```

PART 1 — CREATE RUNPOD ACCOUNT AND LAUNCH GPU POD

~20 minutes — do this once

1.1 Create RunPod Account and Add Credits

~5 minutes

~5 minutes

- Go to **runpod.io** and click **Sign Up**
- Enter your email and password and verify your email
- Click **Billing** in the left sidebar
- Click **Add Payment Method** and enter your credit card
- Click **Add Credits** and add **$30** to start

Why $30?

- Full training run costs $10–20 at $1.89/hr
- Extra $10 covers setup time, smoke test, and any mistakes
- Unused credits never expire — safe to over-fund

1.2 Add Your SSH Key to RunPod

~3 minutes

~3 minutes

Before launching a pod, add your SSH key so you can connect without a password.

- Click **Settings** in the left sidebar (gear icon)
- Click **SSH Public Keys** → **Add SSH Key**
- Paste the entire contents of your public key (from step 0.3)
- Give it a name: **mac-paryaya** → click **Save**

1.3 Launch the A100 Pod

~10 minutes

~10 minutes

■■ **Warning:** Use Secure Cloud, not Community Cloud. Community Cloud pods can be interrupted mid-training with no warning. Secure Cloud pods are stable and guaranteed.

- Click **Secure Cloud** in the left sidebar
- Find **A100 PCIe 40GB** (~$1.89/hr) in the GPU list

If A100 PCIe 40GB is not available, use **A100 SXM 80GB** (~$2.79/hr). It is faster but costs slightly more.

- Click **Deploy** next to your chosen GPU

Pod Configuration — fill in these exact settings:

Setting	Value	Why
Template	RunPod PyTorch 2.2	Pre-installed CUDA 12.1 + Python — saves 30 min setup
`Container Disk`	`30 GB`	`For OS, conda, and pip packages (~15 GB needed)`

Setting	Value	Why
Volume Disk	50 GB	For FLEURS (~3 GB) + checkpoints + HF model cache (~5 GB)
Volume Mount Path	/workspace	Where all data will live — must be /workspace
Expose TCP Ports	22	Required for SSH access

■■ **Warning:** Volume Disk must be at least 50 GB. Google FLEURS downloads ~3 GB of audio. HuggingFace caches Whisper model weights (~3 GB) and tokenizers. Running out of disk mid-training corrupts your checkpoint and wastes GPU hours.

- Click **Deploy** and wait **2–5 minutes** for the pod to reach **Running** status

1.4 Connect to Your Pod via SSH

~3 minutes

~3 minutes

- Once the pod shows **Running**, click **Connect**
- Click **SSH over exposed TCP** — you will see a command like:

```
ssh root@XX.XX.XX.XX -p XXXXX -i ~/.ssh/id_ed25519
```

- Copy that exact command and run it in your Mac terminal

If the connection works you will see:

```
root@A5F8D3B2:~#
```

If Connection Fails

```
# Load your key into the SSH agent first:
ssh-add ~/.ssh/id_ed25519

# Then retry with StrictHostKeyChecking disabled:
ssh root@XX.XX.XX.XX -p XXXXX -i ~/.ssh/id_ed25519 -o StrictHostKeyChecking=no
```

1.5 Verify the GPU is Working

~2 minutes

~2 minutes

Run these commands on the **RunPod terminal** (after SSH-ing in):

```
nvidia-smi
```

Expected: shows **A100-PCIE-40GB** with 40960 MiB memory.

```
python3 -c "import torch; print('CUDA:', torch.cuda.is_available()); "\
        "print('GPU:', torch.cuda.get_device_name(0))"
```

Expected output:

```
CUDA: True
GPU: NVIDIA A100-PCIE-40GB
```

If CUDA shows False, reinstall PyTorch with CUDA support:

```
pip uninstall torch torchaudio -y
pip install torch==2.2.0 torchaudio==2.2.0 \
    --index-url https://download.pytorch.org/whl/cu121
```

PART 2 — SET UP THE POD

~15 minutes — runs mostly automatically

2.1 Move HuggingFace Cache to the Persistent Volume

~1 minute

~1 minute

■■ **Warning:** By default HuggingFace downloads go to ~/.cache/ on the container disk, which is wiped on pod restart. This command moves the cache to /workspace (the persistent volume) so downloads survive disconnection. Run it before anything else.

```
echo 'export HF_HOME=/workspace/.cache/huggingface' >> ~/.bashrc
echo 'export HF_DATASETS_CACHE=/workspace/.cache/huggingface/datasets' >> ~/.bashrc
source ~/.bashrc
mkdir -p /workspace/.cache/huggingface/datasets
```

2.2 Clone the Paryaya Repository

~2 minutes

~2 minutes

```
cd /workspace
git clone https://github.com/asticrat/paryaya.git
cd paryaya
```

Verify the key files are present:

```
ls scripts/
# Should include: finetune_whisper.py  setup_runpod_whisper.sh

ls configs/
# Should include: finetune_whisper.yaml
```

2.3 Run the Setup Script

~10 minutes

~10 minutes

```
bash scripts/setup_runpod_whisper.sh
```

The script installs: git, ffmpeg, libsndfile1, tmux, transformers, datasets, accelerate, evaluate, jiwer, librosa, soundfile, wandb, pyyaml.

Expected ending output:

```
■ System deps installed
■ Repo ready at /workspace/paryaya
■ Python deps installed
CUDA available : True
GPU            : NVIDIA A100-PCIE-40GB
VRAM           : 40.1 GB
```

If a package fails to install, run manually:

```
pip install "transformers>=4.43" "datasets>=2.21" "accelerate>=0.33" \
    "evaluate>=0.4" "jiwer>=3.0" soundfile librosa wandb pyyaml \
    "huggingface-hub>=0.24"
pip install -e .
```

2.4 Set W&B; Key (Optional)

~1 minute

~1 minute

■ HF_TOKEN is NOT required. Google FLEURS is fully public and downloads without authentication. Only set WANDB_API_KEY if you want real-time charts in your browser.

```
# WandB key (optional — for real-time training charts):
export WANDB_API_KEY=abcdef1234567890abcdef1234567890abcdef12
export WANDB_PROJECT=paryaya-whisper

# Save so it persists if pod restarts:
echo "export WANDB_API_KEY=$WANDB_API_KEY" >> ~/.bashrc
echo "export WANDB_PROJECT=paryaya-whisper" >> ~/.bashrc

# Verify it was set:
echo "WANDB: $([ -n \"$WANDB_API_KEY\" ] && echo SET || echo not set)"
# If not using W&B, that's fine — training works without it
```

2.5 Log In to W&B;

~2 minutes (skip if not using W&B;)

~2 minutes (skip if not using W&B;)

```
wandb login
# Paste your API key when prompted, then press Enter

wandb status
# Should show: Currently logged in as: YOUR_USERNAME
```

If you see *wandb: ERROR*, your key is wrong. Get a new one at wandb.ai/settings → API Keys.

2.6 Final Readiness Check

~2 minutes

~2 minutes

Run this block as-is. All lines must show ■ before training:

```
python3 - <<'EOF'
import sys, os, shutil
errors = []

import torch
if not torch.cuda.is_available():
    errors.append("CUDA not available")
else:
    gb = torch.cuda.get_device_properties(0).total_memory / 1e9
    print(f"■  GPU: {torch.cuda.get_device_name(0)} ({gb:.1f} GB)")

try:
    import transformers; print(f"■  transformers: {transformers.__version__}")
except: errors.append("transformers not installed")

try:
    import datasets; print(f"■  datasets: {datasets.__version__}")
except: errors.append("datasets not installed")

# HF_TOKEN not required for FLEURS — just note if set
token = os.getenv("HF_TOKEN", "")
```

```
if token.startswith("hf_"): print(f"■■   HF_TOKEN: set (optional, not needed for FLEURS)")
else: print("■■   HF_TOKEN: not set (OK — FLEURS is public)")

free_gb = shutil.disk_usage("/workspace").free / 1e9
if free_gb < 15: errors.append(f"Low disk: {free_gb:.1f} GB free (need 15+)")
else: print(f"■  Disk: {free_gb:.1f} GB free")

if errors:
    print("\n■  FIX THESE:")
    for e in errors: print(f"   - {e}")
    sys.exit(1)
else: print("\n■  All checks passed!")
EOF
```

PART 3 — SMOKE TEST

~5 minutes — do not skip this

■■ **Warning:** The smoke test runs the full training pipeline on only 8 samples and 2 optimizer steps. It downloads a tiny slice of FLEURS, preprocesses it, and confirms the model trains and saves. Finding a bug here costs 5 minutes. Finding it after 3 hours of GPU time costs money and time that cannot be recovered.

3.1 Run the Smoke Test

~5 minutes

~5 minutes

```
cd /workspace/paryaya

python3 scripts/finetune_whisper.py \
    --config configs/finetune_whisper.yaml \
    --smoke_test
```

Expected Output (Normal)

```
Loading processor from openai/whisper-medium ...
Downloading model.safetensors: 100%|■■■■■■■■■| 1.52G/1.52G ...

Loading google/fleurs (ne_np) ...
Downloading data: 100%|■■■■■■■■■| ...

  ■ Smoke test mode — 2 steps only
Pre-processing audio + tokenising transcripts ...
  train=8  eval=4

Loading openai/whisper-medium weights ...

■ Training on cuda | steps=2 | fp16=True
   Checkpoint dir: checkpoints/whisper-medium-nepali

{'loss': 8.2341, 'learning_rate': ..., 'epoch': ...}
{'eval_loss': ..., 'eval_wer': ..., ...}

■ Training complete.
   Best model saved → checkpoints/whisper-medium-nepali/best
```

■ The smoke test passes if you see '■ Training on cuda' and '■ Training complete.' with no Python traceback.

Smoke Test Error Reference

Error	Cause	Fix
CUDA out of memory	Batch size too large for this GPU	Edit config: per_device_train_batch_size: 8 eval_batch_size: 4
`No module named 'evaluate'`	`evaluate package not installed`	`pip install evaluate>=0.4 jiwer`

Error	Cause	Fix
No space left on device	Volume disk too small	Need at least 15 GB free on /workspace — recreate pod with 50 GB
ConnectionError / Timeout loading FLEURS	Network blip on RunPod	Rerun the same command — HF datasets auto-resumes partial downloads
KeyError: 'transcription'	Wrong text_column in config	Verify configs/finetune_whisper.yaml has text_column: transcription

Fix any error and rerun the smoke test before proceeding to full training.

PART 4 — FULL TRAINING

~3–6 hours unattended — start before bed or when you can leave it running

4.1 Start a tmux Session

~1 minute

~1 minute

tmux keeps training running even after you close your SSH connection. Without it, disconnecting your terminal kills training.

```
# Create a persistent session named 'train':
tmux new -s train
```

You are now inside tmux. The green bar at the bottom of your terminal confirms you are in the session.

4.2 Launch Full Training

~3–6 hours

~3–6 hours

```
cd /workspace/paryaya

python3 scripts/finetune_whisper.py \
    --config configs/finetune_whisper.yaml
```

Training first downloads Google FLEURS Nepali (first run only, ~3 GB, 3–5 minutes on RunPod), then preprocesses the audio, and begins training.

What You Will See

```
Loading processor from openai/whisper-medium ...
Loading google/fleurs (ne_np) ...
Pre-processing audio + tokenising transcripts ...
  train=3,081  eval=579

Loading openai/whisper-medium weights ...

■ Training on cuda | steps=3000 | fp16=True
   Checkpoint dir: checkpoints/whisper-medium-nepali

{'loss': 8.4123, 'learning_rate': 2e-08, 'epoch': 0.09}
{'loss': 7.8234, 'learning_rate': 4e-07, 'epoch': 0.18}
...improving slowly over hours...
```

Training Progress — What to Expect

Steps	Loss	WER	What is Happening
1–100	7–9	80–95%	Model aligning to Devanagari output — looks terrible, completely normal
100–500	5–7	50–80%	Learning Nepali phoneme patterns — WER drops fast
500–2000	2–5	20–50%	Real words forming, model gains vocabulary
2000–3500	1.5–3	15–30%	Fine-tuning word boundaries and context
2000–3000	0.8–2	8–22%	Final polish — target is WER below 20%

Do Not Panic at High Early WER

- WER of 80–95% in the first 100 steps is completely normal
- Whisper already understands Nepali from pre-training — fine-tuning just aligns it
- Loss dropping from 8 to 7 in the first 50 steps confirms training is working
- Only worry if loss is still above 7.5 at step 300+

4.3 Detach from tmux and Leave Training Running

~30 seconds

~30 seconds

Once training is running and you see loss values printing, detach:

```
# Press Ctrl+B, then D  (hold Ctrl, press B, release both, then press D)
```

You will see:

```
[detached (from session train)]
```

Training continues on the pod. You can now safely close your terminal, turn off your laptop, or go to sleep.

4.4 Monitor Training Progress

Ongoing

Ongoing

Option A — W&B; Dashboard (recommended)

Go to **wandb.ai/YOUR_USERNAME/paryaya-whisper** in your browser. You will see live charts for:

- **train/loss** — should decrease steadily throughout training
- **eval/wer** — the key metric, target below 0.20 (20%) by end
- **eval/loss** — decreases with small bumps, that is normal

Option B — From the Terminal

```
# Reconnect to pod and reattach:
ssh root@XX.XX.XX.XX -p XXXXX -i ~/.ssh/id_ed25519
tmux attach -t train

# In a second SSH connection — check GPU is being used (should be 85-99%):
watch -n 5 nvidia-smi

# Check checkpoints are being saved:
ls -lht /workspace/paryaya/checkpoints/whisper-medium-nepali/
# Expect: checkpoint-500  checkpoint-1000  checkpoint-1500  ... updating regularly
```

4.5 Reconnecting After Your SSH Drops

~2 minutes

~2 minutes

Training is still running (tmux keeps it alive). Simply reconnect:

```
ssh root@XX.XX.XX.XX -p XXXXX -i ~/.ssh/id_ed25519
tmux attach -t train
```

If the Pod Restarted (tmux session gone)

Check if checkpoints were saved to the persistent volume:

```
ls /workspace/paryaya/checkpoints/whisper-medium-nepali/
# If checkpoint folders exist, resume from them:
```

```
source ~/.bashrc          # reload WANDB keys
tmux new -s train
cd /workspace/paryaya

# The trainer automatically detects the latest checkpoint
# in the output_dir and resumes from it:
python3 scripts/finetune_whisper.py --config configs/finetune_whisper.yaml
```

4.6 Early Stopping Behaviour

Automatic

Automatic

The config includes **early_stopping_patience: 5**. If validation WER does not improve for 5 consecutive evaluations (5 × 500 steps = 2500 steps without improvement), training stops automatically and saves the best checkpoint.

■ If training stops before step 4000, that is fine. Early stopping found the optimal point. The best model is saved.

4.7 When Training Finishes

~2 minutes

~2 minutes

You will see the final output:

```
■ Training complete.
   Best model saved → checkpoints/whisper-medium-nepali/best

   Deploy to Paryaya API:
   export ASR_BACKEND=whisper
   export WHISPER_MODEL_PATH=checkpoints/whisper-medium-nepali/best
   uvicorn paryaya.api.main:app --host 0.0.0.0 --port 8000
```

Check the best WER achieved during training:

```
cat /workspace/paryaya/checkpoints/whisper-medium-nepali/trainer_state.json \
  | python3 -c "
import sys, json
s = json.load(sys.stdin)
best = min(s['log_history'], key=lambda x: x.get('eval_wer', 999))
print(f\"Best WER: {best.get('eval_wer', 'N/A'):.1%} at step {best.get('step','?')}\")
"
```

Interpreting Your WER

- WER below 20% — Excellent result, model is production-ready
- WER 20–35% — Good result, noticeably better than baseline Whisper on Nepali
- WER 35–50% — Partial success — consider more training steps or data
- WER above 50% — Something went wrong, check that dataset was loaded correctly

PART 5 — DOWNLOAD MODEL AND TERMINATE POD

~15 minutes — do not skip any step

■■ **Warning:** Once you terminate the pod, the volume data is permanently deleted. Download everything listed below before clicking Terminate. Double-check each file downloaded successfully before terminating.

5.1 Verify the Model Was Saved on the Pod

~2 minutes

~2 minutes

Run on the RunPod terminal:

```
ls -lh /workspace/paryaya/checkpoints/whisper-medium-nepali/best/
```

Expected — these files must all be present:

```
config.json
generation_config.json
model.safetensors       (~1.5 GB)
preprocessor_config.json
special_tokens_map.json
tokenizer.json
tokenizer_config.json
vocab.json
added_tokens.json
merges.txt
```

If the **best/** folder is empty or missing, save manually:

```
python3 - <<'EOF'
import json
from pathlib import Path
from transformers import WhisperForConditionalGeneration, WhisperProcessor

state = json.loads(Path("checkpoints/whisper-medium-nepali/trainer_state.json").read_text())
best = state.get("best_model_checkpoint", "checkpoints/whisper-medium-nepali/checkpoint-500")
print(f"Saving from: {best}")

model = WhisperForConditionalGeneration.from_pretrained(best)
proc  = WhisperProcessor.from_pretrained(best)
Path("checkpoints/whisper-medium-nepali/best").mkdir(parents=True, exist_ok=True)
model.save_pretrained("checkpoints/whisper-medium-nepali/best")
proc.save_pretrained("checkpoints/whisper-medium-nepali/best")
print("Done")
EOF
```

5.2 Download the Model to Your Mac

~5–10 minutes

~5–10 minutes

Open a **new terminal on your Mac** (leave the RunPod SSH session open):

```
# Create a folder to receive the model:
mkdir -p ~/paryaya_model

# Download the entire best/ folder (~1.5 GB, takes 5-10 minutes)
# Replace XX.XX.XX.XX and XXXXX with your actual pod IP and port:
```

```
scp -P XXXXX -r \
    root@XX.XX.XX.XX:/workspace/paryaya/checkpoints/whisper-medium-nepali/best/ \
    ~/paryaya_model/whisper-medium-nepali/
```

Verify the download:

```
ls -lh ~/paryaya_model/whisper-medium-nepali/
# model.safetensors should be ~1.5 GB

du -sh ~/paryaya_model/
# Should show ~1.5–1.6 GB total
```

Quick load test to confirm the file is not corrupted:

```
cd ~/paryaya_model
python3 - <<'EOF'
from transformers import WhisperForConditionalGeneration, WhisperProcessor
m = WhisperForConditionalGeneration.from_pretrained("whisper-medium-nepali")
print(f"■ Loaded: {m.num_parameters():,} params | lang: {m.generation_config.language}")
EOF
# Expected: ■ Loaded: 307,198,976 params | lang: nepali
```

5.3 Copy the Model into Your Paryaya Project

~1 minute

~1 minute

```
cp -r ~/paryaya_model/whisper-medium-nepali/ \
    /Users/yaxzyra/Documents/asti-lab/paryaya/checkpoints/whisper-medium-nepali/

echo "Model is now at:"
ls /Users/yaxzyra/Documents/asti-lab/paryaya/checkpoints/whisper-medium-nepali/
```

5.4 Terminate the RunPod Pod

~3 minutes

~3 minutes

Only do this after verifying the download in steps 5.2 and 5.3.

- Go to **runpod.io** → **My Pods**
- Find your pod — it shows as **Running**
- Click the **three dots (...)** menu on the right side of your pod
- Click **Terminate Pod**
- Confirm by clicking **Terminate** in the dialog
- The pod disappears from the list and billing stops immediately

■■ **Warning:** Do NOT just stop the pod (the Stop button). A stopped pod still charges for volume storage. Use Terminate to stop all billing immediately.

After terminating, confirm your total spend: **runpod.io** → **Billing** → **Usage**. Should be under $20.

PART 6 — DEPLOY VIA PARYAYA API

~10 minutes

6.1 Test Locally First

~5 minutes

~5 minutes

```
cd /Users/yaxzyra/Documents/asti-lab/paryaya
source .venv/bin/activate

export ASR_BACKEND=whisper
export WHISPER_MODEL_PATH=/Users/yaxzyra/Documents/asti-lab/paryaya/checkpoints/whisper-medium-nepali

uvicorn paryaya.api.main:app --host 0.0.0.0 --port 8000
```

Expected startup logs:

```
INFO  Starting Paryaya API | backend=whisper device=mps
INFO  Whisper backend loaded: .../checkpoints/whisper-medium-nepali
INFO  Application startup complete.
```

In a new terminal, test the API:

```
curl http://localhost:8000/health
# Expected: {"status":"ok","model_loaded":true,"backend":"whisper",...}

curl -X POST http://localhost:8000/v1/transcribe \
    -H "Authorization: Bearer sk-paryaya-testkey123" \
    -F "file=@/path/to/nepali_audio.wav"
```

6.2 Docker Deployment

~5 minutes

~5 minutes

Add these environment variables to your **.env** file or docker-compose.yml environment section:

```
ASR_BACKEND=whisper
WHISPER_MODEL_PATH=/app/checkpoints/whisper-medium-nepali
```

In **docker/docker-compose.yml**, add a volume mount if not already present:

```
services:
  api:
    environment:
      - ASR_BACKEND=whisper
      - WHISPER_MODEL_PATH=/app/checkpoints/whisper-medium-nepali
    volumes:
      - ./checkpoints:/app/checkpoints
```

Deploy:

```
docker compose -f docker/docker-compose.yml up -d
docker compose -f docker/docker-compose.yml logs -f api
```

TROUBLESHOOTING

Training Problems

Loss stuck above 7.5 after step 200

Steps 1–100 are always slow. If still stuck at step 200+, check:

```
# Verify fp16 is actually being used:
grep 'fp16' configs/finetune_whisper.yaml
# Should show: fp16: true

# Check GPU memory is being used (should be 25–38 GB):
nvidia-smi
# If under 10 GB, fp16 is falling back to fp32
```

CUDA out of memory

```
# Reduce batch size in configs/finetune_whisper.yaml:
# Change: per_device_train_batch_size: 16  →  8
# Change: per_device_eval_batch_size: 8   →  4
# Change: gradient_accumulation_steps: 2  →  4  (keeps effective batch = 32)
# Save and rerun training
```

W&B; not logging / wandb: ERROR

```
wandb login --relogin   # re-enter your API key

# Or disable W&B entirely:
export WANDB_MODE=disabled
python3 scripts/finetune_whisper.py --config configs/finetune_whisper.yaml
```

evaluate.load('wer') fails

```
pip install evaluate>=0.4 jiwer>=3.0
python3 -c "import evaluate; m = evaluate.load('wer'); print('OK')"
```

Training is very slow (>5 seconds per step)

The first run includes audio preprocessing via the map() function which is slow. Once preprocessing finishes and actual training steps begin, each step should take 0.3–0.8 seconds. If steps are still slow after 30 minutes, check GPU utilisation with nvidia-smi.

Dataset Problems

ConnectionError or timeout when downloading FLEURS

Google FLEURS downloads from HuggingFace CDN. If the download fails partway:

```
# Just rerun the same training command — HuggingFace datasets
# automatically resumes partial downloads from where they stopped.
python3 scripts/finetune_whisper.py --config configs/finetune_whisper.yaml --smoke_test
```

KeyError: 'transcription'

The text column name in the config does not match the dataset. Verify:

```
grep text_column configs/finetune_whisper.yaml
# Should show: text_column: "transcription"
```

```
# If missing, add it:
echo '  text_column: "transcription"' >> configs/finetune_whisper.yaml
```

Dataset download is slow

FLEURS Nepali is ~3 GB and should download in 3–8 minutes on RunPod. If slower than 15 minutes, there may be a RunPod network issue — wait it out rather than restarting.

Connection Problems

SSH disconnects frequently

```
# Add keepalive flags to your SSH command:
ssh root@XX.XX.XX.XX -p XXXXX -i ~/.ssh/id_ed25519 \
    -o ServerAliveInterval=60 \
    -o ServerAliveCountMax=10
```

GitHub SSH: connection timed out on port 22

```
# Add this to ~/.ssh/config on your Mac:
nano ~/.ssh/config

# Add these lines:
Host github.com
  Hostname ssh.github.com
  Port 443
  AddKeysToAgent yes
  IdentityFile ~/.ssh/id_ed25519
```

Pod IP/port changed after reconnect

Pod IPs and ports can change after a restart. Always get the current SSH command from **runpod.io** → **My Pods** → **Connect** rather than using a saved command.

API Problems

API returns 503 model_loaded: false

```
# Check that WHISPER_MODEL_PATH points to a valid directory:
ls $WHISPER_MODEL_PATH
# Should show model.safetensors and config.json

# Check the API startup logs:
docker compose -f docker/docker-compose.yml logs api | tail -20
```

Latency is high (>3 seconds per request)

On CPU (MPS on Mac), Whisper medium takes 2–4 seconds per clip. On a server without GPU, this is expected. On GPU it should be 200–400ms. Set BEAM_WIDTH=1 in your environment for faster (but slightly less accurate) decoding.

QUICK REFERENCE AND COST SUMMARY

Commands You Will Use Most

Action	Command
Connect to pod	`ssh root@XX.XX.XX.XX -p XXXXX -i ~/.ssh/id_ed25519`
Reattach to training	tmux attach -t train
Detach from tmux	Ctrl+B, then D
Check GPU	nvidia-smi (in second SSH session)
Check checkpoints	ls -lht /workspace/paryaya/checkpoints/whisper-medium-nepali/
Check disk	df -h /workspace
Reload env vars	source ~/.bashrc
Run smoke test	`python3 scripts/finetune_whisper.py --config configs/finetune_whisper.yaml --smoke_test`
Download model (Mac)	scp -P XXXXX -r root@XX.XX.XX.XX:/workspace/paryaya/checkpoints/whisper-medium-nepali/best/ ~/paryaya_model/whisper-medium-nepali/
Start API (local)	`export ASR_BACKEND=whisper && uvicorn paryaya.api.main:app --port 8000`
Check API health	`curl http://localhost:8000/health`

Cost Summary

Phase	Duration	Cost at $1.89/hr
Account setup (no pod running)	30 min	Free
Pod setup + smoke test	30 min	~$0.95
Dataset download + preprocessing	30 min	~$0.95
Full training (3000 steps, FLEURS)	2–3 hrs	~$3.78–5.67
Download model + terminate	20 min	~$0.63
Total	4.5–7 hrs	~$8–12

Budget Check

- A100 PCIe 40GB runs at ~$1.89/hr on RunPod Secure Cloud (price may vary)
- A6-hour full run (including setup) costs ~$11.34
- With $30 credit loaded, you have 4x more than needed — no risk of running out
- Unused RunPod credits never expire

Environment Variables for Deployment

Variable	Value	Required?
ASR_BACKEND	whisper	Yes
WHISPER_MODEL_PATH	/path/to/checkpoints/whisper-medium-nepali	Yes
MAX_AUDIO_MB	50 (default)	No
BEAM_WIDTH	5 (lower = faster, slightly less accurate)	No
LOG_LEVEL	INFO	No

File Locations After Completion

File/Folder	Location
Fine-tuned model	/Users/yaxzyra/Documents/asti-lab/paryaya/checkpoints/whisper-medium-nepali/
Training config	/Users/yaxzyra/Documents/asti-lab/paryaya/configs/finetune_whisper.yaml
Fine-tuning script	/Users/yaxzyra/Documents/asti-lab/paryaya/scripts/finetune_whisper.py
RunPod setup script	/Users/yaxzyra/Documents/asti-lab/paryaya/scripts/setup_runpod_whisper.sh
Whisper API backend	/Users/yaxzyra/Documents/asti-lab/paryaya/src/paryaya/inference/whisper_backend.py
This guide (markdown)	/Users/yaxzyra/Documents/asti-lab/paryaya/docs/runpod_finetune_guide.md

Paryaya — ■■■■■■ — many voices, one understanding.
github.com/asticrat/paryaya